RapidResolve: Concepts Used (Java, DS, DBMS)

Ultra-short mapping from concept → where/why used. Print this page to PDF.

Java OOP

- **Inheritance**: `Admin`, `Officer`, `Citizen` extend `User` — role-specific menus (`Admin.java`, `Officer.java`, `Citizen.java`).
- **Abstraction**: `User` is `abstract` with `showMenu(Scanner)` forcing role implementations (`User.java`).
- **Polymorphism/Overriding**: `showMenu` overridden in each role; invoked based on concrete type after login (`MainApplication.loginFlow`).
- **Encapsulation**: private fields + getters in domain classes (`Complaint.java`, `Crime.java`, `User.java`).
- **Constructor overloading**: multiple constructors for `User` and `Officer` to support different contexts.
- **Static utility classes**: `CLIUtils`, `DBConnection`, `ActionTracker`, `HashingForPassword`, `ReportGenerator`, `QRGenerator`.

Java Core / Utilities

- **Exception handling**: try/catch, try-with-resources across DB/file ops (e.g., `ReportGenerator`, `Admin.showFeedbackRatingsSummary`).
- **Console I/O & formatting**: `Scanner` input, ANSI colors, boxed menus (`CLIUtils.java`).
- **File I/O**: `FileWriter` / `BufferedWriter` for TXT reports + OTP files (`ReportGenerator`, `MainApplication`); `Desktop.open` for QR images (`QRGenerator`).
- **Validation & Regex**: reusable prompts with regex checks; username/email validation (`CLIUtils.promptString`, `MainApplication.isValidUsername/normalizeEmail`).
- **Random/OTP**: `Random` for OTP and temp passwords (`MainApplication`, `OfficerManager`).
- **Date/Time**: `java.time.LocalDate` for report timestamps and filing dates.

Collections / Data Structures

- **BST**: sorting complaints by `complaint_id` via in-order traversal — `ComplaintBST.insert`, `inOrder`; populated in `ComplaintManager.viewComplaintsByUser`.
- **Deque (sliding window)**: track last 3 OTP failures/user and lock for 5 minutes — `Deque<Long>` with `LinkedList` (`OTPAttemptTracker`).

- **Map**: user→attempts and user→unlockTime — `Map<String, ...>` (`OTPAttemptTracker`).
- **ArrayList**: accumulate IDs for bulk delete during archiving (`ReportGenerator.archiveOldComplaints`).
- **Arrays**: char array for password generation (`OfficerManager.generateRandomPassword`).

Algorithms

- **In-order traversal**: outputs complaints sorted by ID (`ComplaintBST.inOrder`).
- **Sliding time window**: OTP lockout using timestamps queue comparison (`OTPAttemptTracker`).

DBMS / JDBC (MySQL)

- **JDBC primitives**: `Connection` , `PreparedStatement` , `ResultSet` , `Statement` used throughout (`DBConnection` , managers).
- **Prepared statements**: parameterized queries to prevent SQL injection (all managers).
- **Transactions**: `setAutoCommit(false)` , `commit` , `rollback` — atomic inserts/updates (`ComplaintManager.fileComplaint` , `OfficerManager.addOfficerWithEmail` , `ReportGenerator.archiveOldComplaints`).
- **CRUD**: INSERT (complaints/crimes/feedback/suggestions), SELECT (views), UPDATE (status/profile/password), DELETE/TRUNCATE (logs, users, related data).
- **Joins**:
    - INNER JOIN: complaint + user + officer for QR content (`QRGenerator.generateQRCode`).
    - LEFT JOIN: complaints↔users for report (`ReportGenerator.generateAllComplaintsReport`).
    - LEFT JOIN: load distribution base for assignment (`OfficerManager.assignOfficer`).
- **Aggregations**: `COUNT(*)` totals; `GROUP BY + ORDER BY` for ratings summary (`Admin.showFeedbackRatingsSummary`).
- **Ordering & limits**: `ORDER BY filed_on DESC` , `ORDER BY complaint_id DESC LIMIT 1` (latest complaint).
- **Date functions**: MySQL `DATE_SUB` , `CURDATE()` for snapshots and archiving (`CrimeManager.showSnapshot` , `ReportGenerator.archiveOldComplaints`).
- **Generated keys**: `RETURN_GENERATED_KEYS` to capture new user ID (`UserManager.registerUser`).
- **Updatable ResultSet**: migrate plaintext→SHA-256 passwords in-place (`HashingForPassword`).
- **Metadata**: `DatabaseMetaData` to show DB properties (`Admin.viewSystemPaths`).

Security

- **Password hashing**: SHA-256 using `MessageDigest` (`HashingForPassword.hashPassword` ; consumed in `MainApplication` , managers).
- **First-login enforcement**: `must_change_password` flag checked for officers; update via `OfficerManager.updatePassword` .
- **OTP throttle/lockout**: time-windowed failure tracking + 5-minute lock (`OTPAttemptTracker`).
- **Credentials delivery**: JavaMail SMTP with TLS to send temp creds (`OfficerManager.sendEmail`).

External Libraries / Integrations

- **QR Codes**: ZXing (`MultiFormatWriter` , `MatrixToImageWriter`) to generate PNGs for complaint/user (`QRGenerator`).
- **Email**: JavaMail (`javax.mail`) for sending officer credentials (`OfficerManager`).

Reporting & Archival

- **TXT reporting**: system-wide and per-officer complaint reports (`ReportGenerator.generateAllComplaintsReport` , `generateOfficerComplaintsReport`).
- **Archival**: export resolved/closed >=1 year complaints to TXT and bulk-delete via `IN` list (`ReportGenerator.archiveOldComplaints`).

Application Architecture

- **Service/Manager layer**: `UserManager` , `OfficerManager` , `ComplaintManager` , `CrimeManager` encapsulate DB logic (DAO-lite).
- **Audit logging**: `ActionTracker` writes user actions to `actions_log` table; view/clear via admin.

Tip: Open this file in your browser and use “Print → Save as PDF”.